

the output list if it is). Note that we don't fully instantiate the intermediate stream that results from the `map`. It's exactly as if we had interleaved the logic using a special-purpose loop. For this reason, people sometimes describe streams as "first-class loops" whose logic can be combined using higher-order functions like `map` and `filter`.

Since intermediate streams aren't instantiated, it's easy to reuse existing combinators in novel ways without having to worry that we're doing more processing of the stream than necessary. For example, we can reuse `filter` to define `find`, a method to return just the first element that matches if it exists. Even though `filter` transforms the whole stream, that transformation is done lazily, so `find` terminates as soon as a match is found:

```
def find(p: A => Boolean): Option[A] =
  filter(p).headOption
```

The incremental nature of stream transformations also has important consequences for memory usage. Because intermediate streams aren't generated, a transformation of the stream requires only enough working memory to store and transform the current element. For instance, in the transformation `Stream(1,2,3,4).map(_ + 10).filter(_ % 2 == 0)`, the garbage collector can reclaim the space allocated for the values 11 and 13 emitted by `map` as soon as `filter` determines they aren't needed. Of course, this is a simple example; in other situations we might be dealing with larger numbers of elements, and the stream elements themselves could be large objects that retain significant amounts of memory. Being able to reclaim this memory as quickly as possible can cut down on the amount of memory required by your program as a whole.

We'll have a lot more to say about defining memory-efficient streaming calculations, in particular calculations that require I/O, in part 4 of this book.

5.4 Infinite streams and corecursion

Because they're incremental, the functions we've written also work for *infinite streams*. Here's an example of an infinite `Stream` of 1s:

```
val ones: Stream[Int] = Stream.cons(1, ones)
```

Although `ones` is infinite, the functions we've written so far only inspect the portion of the stream needed to generate the demanded output. For example:

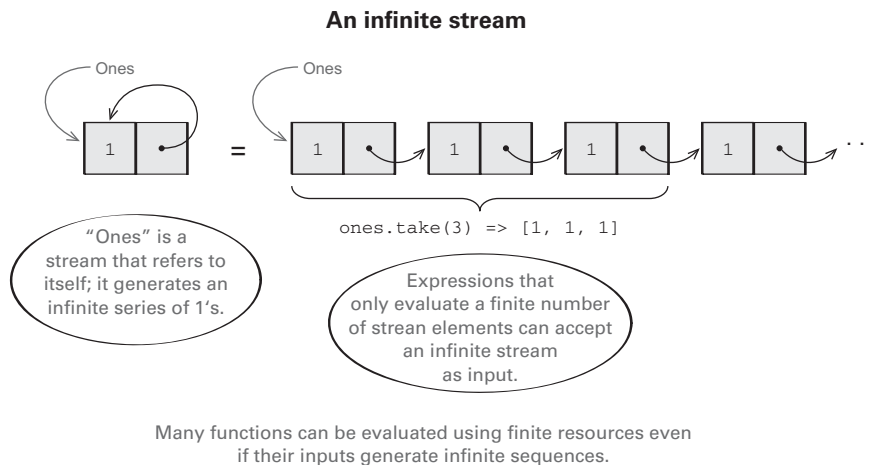
```
scala> ones.take(5).toList
res0: List[Int] = List(1, 1, 1, 1, 1)

scala> ones.exists(_ % 2 != 0)
res1: Boolean = true
```

Try playing with a few other examples:

- `ones.map(_ + 1).exists(_ % 2 == 0)`
- `ones.takeWhile(_ == 1)`
- `ones.forAll(_ != 1)`

In each case, we get back a result immediately. Be careful though, since it's easy to write expressions that never terminate or aren't stack-safe. For example, `ones.forAll(_ == 1)` will forever need to inspect more of the series since it'll never encounter an element that allows it to terminate with a definite answer (this will manifest as a stack overflow rather than an infinite loop).⁶



Let's see what other functions we can discover for generating streams.



EXERCISE 5.8

Generalize `ones` slightly to the function `constant`, which returns an infinite `Stream` of a given value.

```
def constant[A](a: A): Stream[A]
```



EXERCISE 5.9

Write a function that generates an infinite stream of integers, starting from `n`, then `n + 1`, `n + 2`, and so on.⁷

```
def from(n: Int): Stream[Int]
```

⁶ It's possible to define a stack-safe version of `forAll` using an ordinary recursive loop.

⁷ In Scala, the `Int` type is a 32-bit signed integer, so this stream will switch from positive to negative values at some point, and will repeat itself after about four billion elements.

**EXERCISE 5.10**

Write a function `fibs` that generates the infinite stream of Fibonacci numbers: 0, 1, 1, 2, 3, 5, 8, and so on.

**EXERCISE 5.11**

Write a more general stream-building function called `unfold`. It takes an initial state, and a function for producing both the next state and the next value in the generated stream.

```
def unfold[A, S](z: S)(f: S => Option[(A, S)]): Stream[A]
```

`Option` is used to indicate when the `Stream` should be terminated, if at all. The function `unfold` is a very general `Stream`-building function.

The `unfold` function is an example of what's sometimes called a *corecursive* function. Whereas a recursive function consumes data, a corecursive function *produces* data. And whereas recursive functions terminate by recursing on smaller inputs, corecursive functions need not terminate so long as they remain *productive*, which just means that we can always evaluate more of the result in a finite amount of time. The `unfold` function is productive as long as `f` terminates, since we just need to run the function `f` one more time to generate the next element of the `Stream`. Corecursion is also sometimes called *guarded recursion*, and productivity is also sometimes called *cotermination*. These terms aren't that important to our discussion, but you'll hear them used sometimes in the context of functional programming. If you're curious to learn where they come from and understand some of the deeper connections, follow the references in the chapter notes.

**EXERCISE 5.12**

Write `fibs`, `from`, `constant`, and `ones` in terms of `unfold`.⁸

⁸ Using `unfold` to define `constant` and `ones` means that we don't get sharing as in the recursive definition `val ones: Stream[Int] = cons(1, ones)`. The recursive definition consumes constant memory even if we keep a reference to it around while traversing it, while the `unfold`-based implementation does not. Preserving sharing isn't something we usually rely on when programming with streams, since it's extremely delicate and not tracked by the types. For instance, sharing is destroyed when calling even `xs.map(x => x)`.

**EXERCISE 5.13**

Use `unfold` to implement `map`, `take`, `takeWhile`, `zipWith` (as in chapter 3), and `zipAll`. The `zipAll` function should continue the traversal as long as either stream has more elements—it uses `Option` to indicate whether each stream has been exhausted.

```
def zipAll[B](s2: Stream[B]): Stream[(Option[A], Option[B])]
```

Now that we have some practice writing stream functions, let's return to the exercise we covered at the end of chapter 3—a function, `hasSubsequence`, to check whether a list contains a given subsequence. With strict lists and list-processing functions, we were forced to write a rather tricky monolithic loop to implement this function without doing extra work. Using lazy lists, can you see how you could implement `hasSubsequence` by combining some other functions we've already written?⁹ Try to think about it on your own before continuing.

**EXERCISE 5.14**

Hard: Implement `startsWith` using functions you've written. It should check if one `Stream` is a prefix of another. For instance, `Stream(1,2,3) startsWith Stream(1,2)` would be true.

```
def startsWith[A](s: Stream[A]): Boolean
```

**EXERCISE 5.15**

Implement `tails` using `unfold`. For a given `Stream`, `tails` returns the `Stream` of suffixes of the input sequence, starting with the original `Stream`. For example, given `Stream(1,2,3)`, it would return `Stream(Stream(1,2,3), Stream(2,3), Stream(3), Stream())`.

```
def tails: Stream[Stream[A]]
```

We can now implement `hasSubsequence` using functions we've written:

```
def hasSubsequence[A](s: Stream[A]): Boolean =
  tails exists (_ startsWith s)
```

⁹ This small example, of assembling `hasSubsequence` from simpler functions using laziness, is from Cale Gibbard. See this post: <http://lambda-the-ultimate.org/node/1277#comment-14313>.